



National Textile University

Department of Computer Science

Subject: Operating System

Submitted to: Sir Nasir

Submitted by: Kashmir Jamshaid

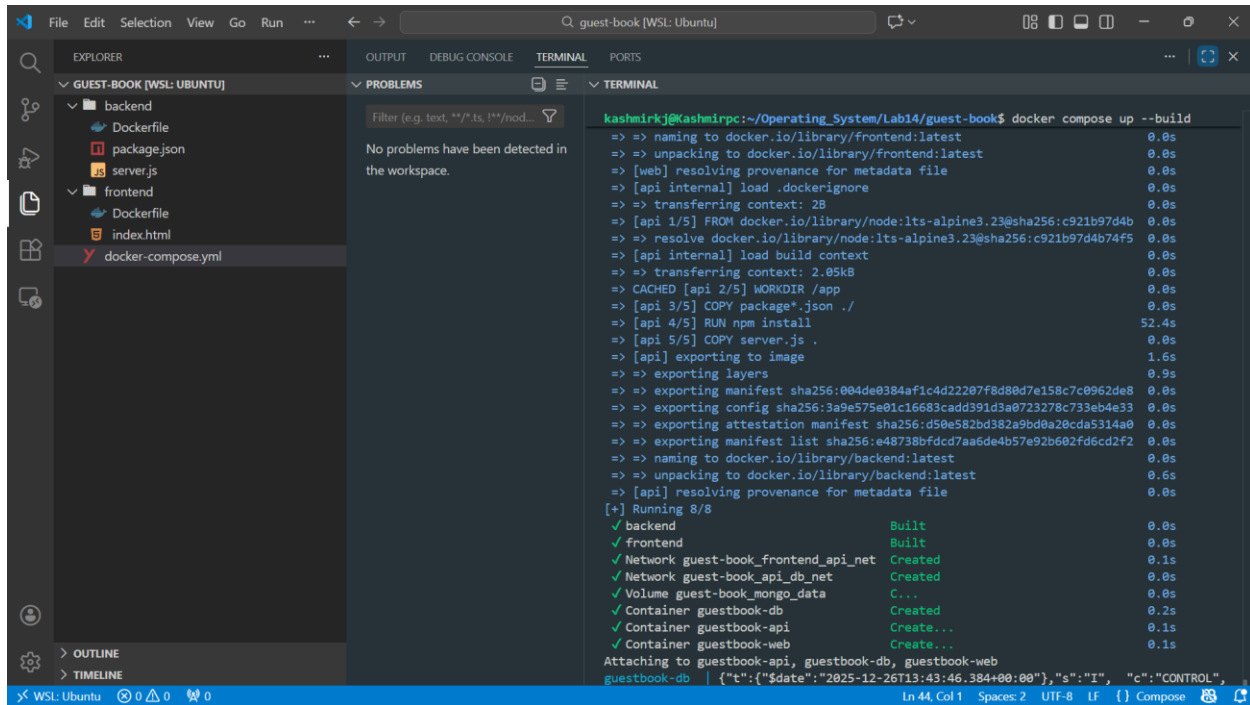
Reg. number: 23-NTU-CS-1167

Lab No 14 (Docker #3)

Semester:5th

Operating Systems – COC 3071

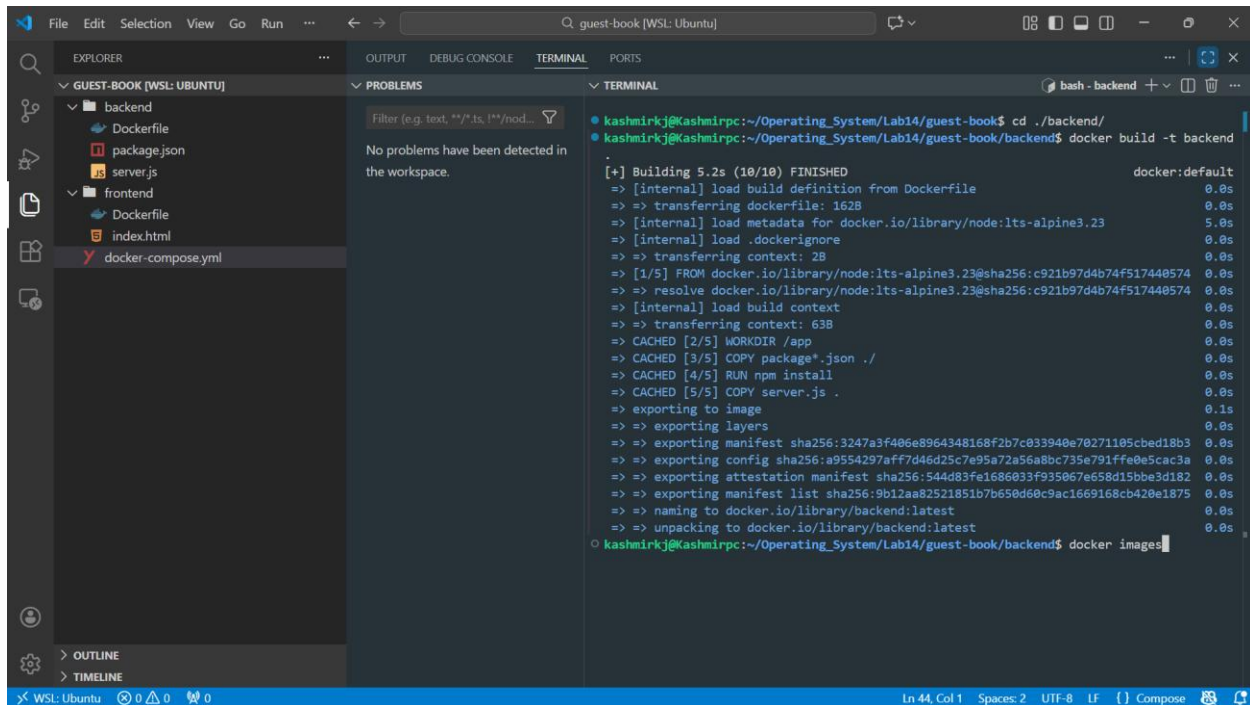
Whole Structure of project :



The screenshot shows the VS Code interface with the Explorer view on the left displaying the project structure: `backend` (containing `Dockerfile`, `package.json`, and `server.js`) and `frontend` (containing `Dockerfile`, `index.html`, and `docker-compose.yml`). The Problems view is empty. The Terminal view shows the output of the command `docker compose up --build`, which builds the `backend` and `frontend` images and creates the necessary Docker containers and networks.

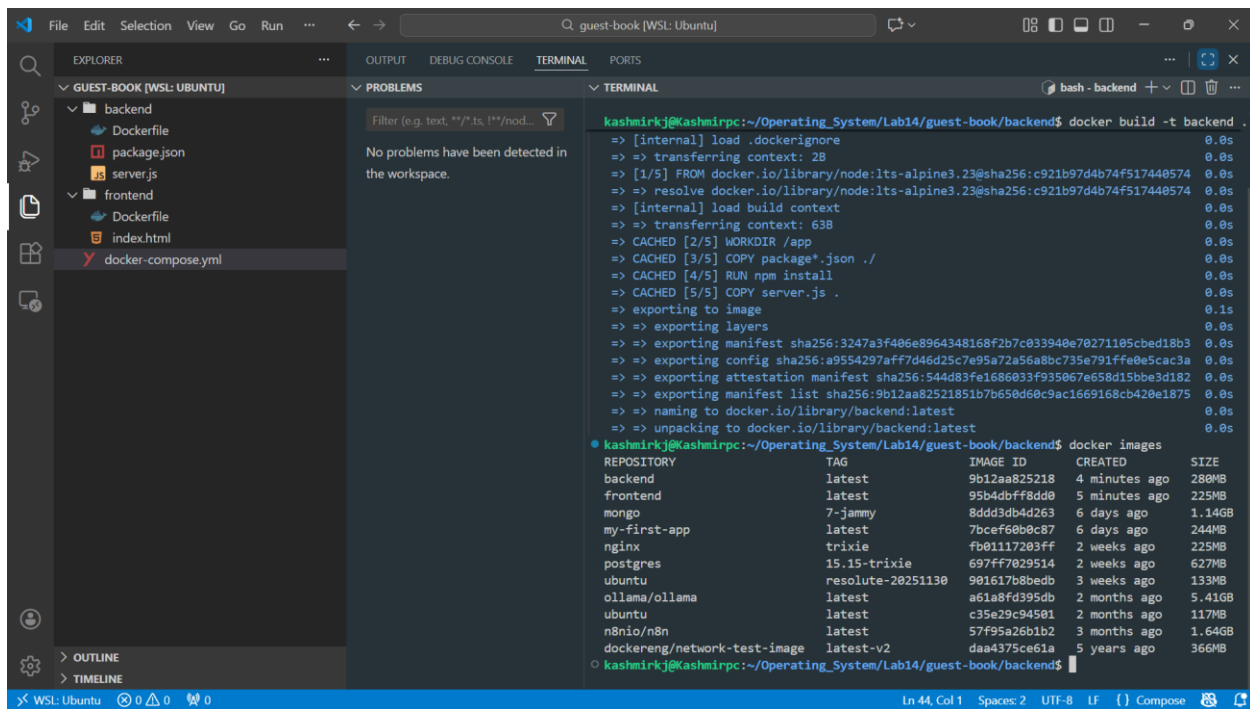
```
kashmirkj@Kashmirpc:~/Operating_System/Lab14/guest-book$ docker compose up --build
=> naming to docker.io/library/frontend:latest 0.0s
=> unpacking to docker.io/library/frontend:latest 0.0s
=> [web] resolving provenance for metadata file 0.0s
=> [api internal] load .dockerignore 0.0s
=> transferring context: 2B 0.0s
=> [api 1/5] FROM docker.io/library/node:lts-alpine3.23@sha256:c921b97d4b 0.0s
=> resolve docker.io/library/node:lts-alpine3.23@sha256:c921b97d4b74f5 0.0s
=> [api internal] load build context 0.0s
=> transferring context: 2.05kB 0.0s
=> CACHED [api 2/5] WORKDIR /app 0.0s
=> [api 3/5] COPY package*.json ./ 0.0s
=> [api 4/5] RUN npm install 52.4s
=> [api 5/5] COPY server.js . 0.0s
=> [api] exporting to image 1.6s
=> exporting layers 0.9s
=> exporting manifest sha256:004de0384af1c4d22207f8d80d7e158c7c0962de8 0.0s
=> exporting config sha256:3a9e575e01c16683cadd391d3a0723278c733eb4e33 0.0s
=> exporting attestation manifest sha256:d50e582bd382a9bd8a20cda5314a0 0.0s
=> exporting manifest list sha256:e48738bfcd7aa6de4b57e92b602fd6cd2f2 0.0s
=> naming to docker.io/library/backend:latest 0.0s
=> unpacking to docker.io/library/backend:latest 0.6s
=> [api] resolving provenance for metadata file 0.0s
[+] Running 8/8
  backend          Built 0.0s
  frontend         Built 0.0s
  Network guest-book_frontend_api_net Created 0.1s
  Network guest-book_api_db_net Created 0.0s
  Volume guest-book_mongo_data C... 0.0s
  Container guestbook-db Created 0.2s
  Container guestbook-api Create... 0.1s
  Container guestbook-web Create... 0.1s
Attaching to guestbook-api, guestbook-db, guestbook-web
guestbook-db | {"t":{"$date":"2025-12-26T13:43:46.384+00:00"},"s":"I", "c":"CONTROL",
```

Backend Image Creation :

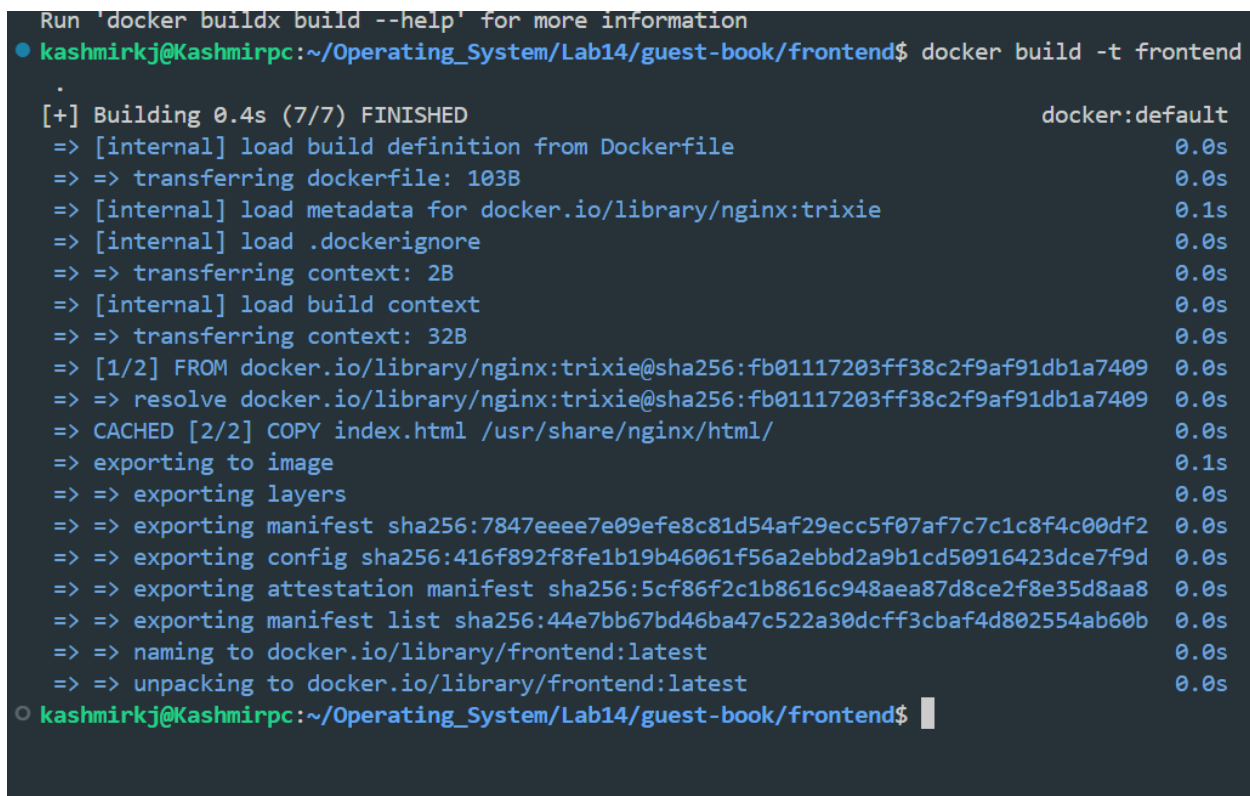


The screenshot shows the VS Code interface with the Explorer view on the left displaying the project structure. The Problems view is empty. The Terminal view shows the output of the command `docker build -t backend`, which builds the `backend` image. The output shows the build process, including the transfer of the Dockerfile, the load of the build context, and the export of the image. The final output shows the image name `backend` and the image ID `sha256:3247a3f406e8964348168f2b7c033940e70271105cbcd18b3`.

```
kashmirkj@Kashmirpc:~/Operating_System/Lab14/guest-book$ cd ./backend/
kashmirkj@Kashmirpc:~/Operating_System/Lab14/guest-book/backend$ docker build -t backend
[+] Building 5.2s (10/10) FINISHED docker:default
=> [internal] load build definition from Dockerfile 0.0s
=> transferring dockerfile: 162B 0.0s
=> [internal] load metadata for docker.io/library/node:lts-alpine3.23 5.0s
=> [internal] load .dockerignore 0.0s
=> transferring context: 2B 0.0s
=> [1/5] FROM docker.io/library/node:lts-alpine3.23@sha256:c921b97d4b74f517440574 0.0s
=> resolve docker.io/library/node:lts-alpine3.23@sha256:c921b97d4b74f517440574 0.0s
=> [internal] load build context 0.0s
=> transferring context: 63B 0.0s
=> CACHED [2/5] WORKDIR /app 0.0s
=> CACHED [3/5] COPY package*.json ./ 0.0s
=> CACHED [4/5] RUN npm install 0.0s
=> CACHED [5/5] COPY server.js . 0.0s
=> exporting to image 0.1s
=> exporting layers 0.0s
=> exporting manifest sha256:3247a3f406e8964348168f2b7c033940e70271105cbcd18b3 0.0s
=> exporting config sha256:a9554297aff7d46d25c7e95a72a56a8bc735e791ffe0e5cac3a 0.0s
=> exporting attestation manifest sha256:544d83fe1686033f935067e658d15bbe3d182 0.0s
=> exporting manifest list sha256:9b12aa82521851b7b650d60c9ac1669168cb420e1875 0.0s
=> naming to docker.io/library/backend:latest 0.0s
=> unpacking to docker.io/library/backend:latest 0.0s
kashmirkj@Kashmirpc:~/Operating_System/Lab14/guest-book/backend$ docker images
```

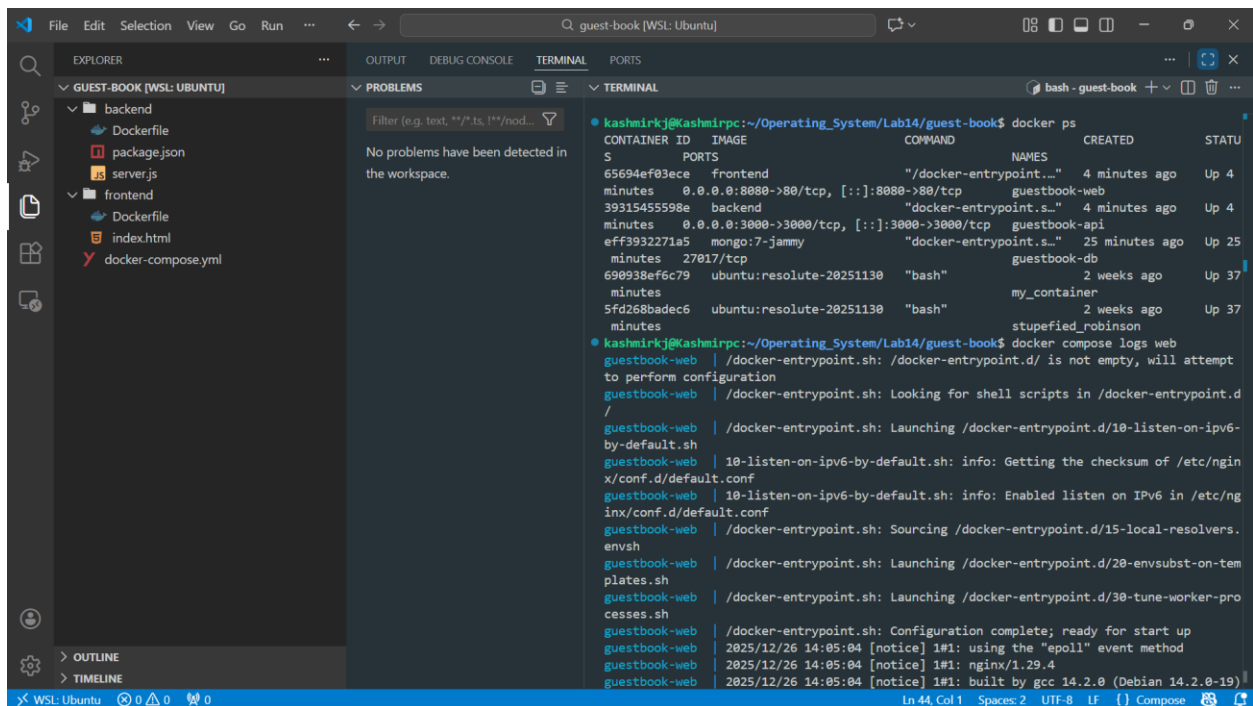


Frontend Image Creation:

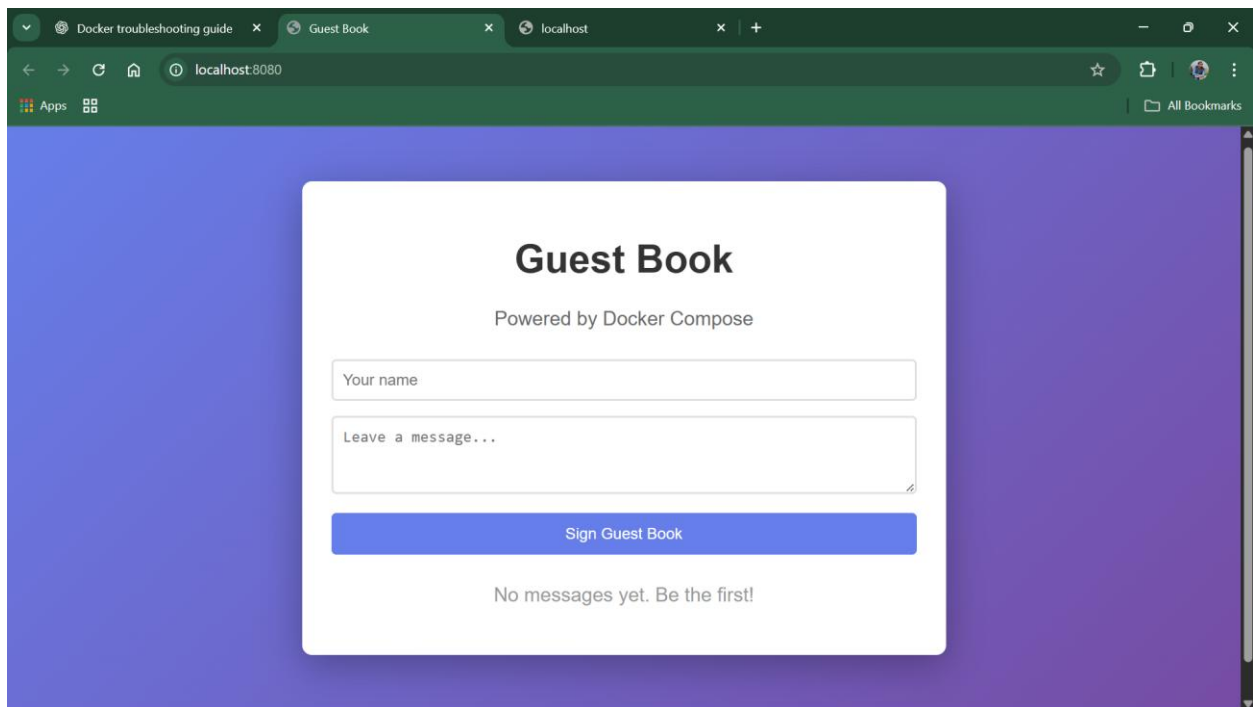


```
kashmirkj@Kashmirpc:~/Operating_System/Lab14/guest-book/frontend$ docker pull mongo:7-jammy
7-jammy: Pulling from library/mongo
Digest: sha256:8ddd3db4d2638eb914cce56284e2f0d6daf140bba31679b2af86f7d790a4c77e
Status: Image is up to date for mongo:7-jammy
docker.io/library/mongo:7-jammy
kashmirkj@Kashmirpc:~/Operating_System/Lab14/guest-book/frontend$
```

Web Page (Frontend):



```
File Edit Selection View Go Run ...  guest-book [WSL: Ubuntu]
EXPLORER
GUEST-BOOK [WSL: UBUNTU]
  backend
    Dockerfile
    package.json
    server.js
  frontend
    Dockerfile
    index.html
    docker-compose.yml
PROBLEMS
No problems have been detected in the workspace.
TERMINAL
bash - guest-book
kashmirkj@Kashmirpc:~/Operating_System/Lab14/guest-book$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS
65694ef03ece   frontend  "/docker-entrypoint..." 4 minutes ago  Up 4
minutes      0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   guestbook-web
39315455598e   backend   "docker-entrypoint.s..." 4 minutes ago  Up 4
minutes      0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp guestbook-api
eff3932271a5   mongo:7-jammy "docker-entrypoint.s..." 25 minutes ago  Up 25
minutes      27017/tcp                                     guestbook-db
690938ef6c79   ubuntu:resolute-20251130 "bash"                  2 weeks ago   Up 37
minutes      5fd268badec6   ubuntu:resolute-20251130 "bash"                  2 weeks ago   Up 37
minutes      stupefied_robinson
kashmirkj@Kashmirpc:~/Operating_System/Lab14/guest-book$ docker compose logs web
guestbook-web | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt
to perform configuration
guestbook-web | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d
/
guestbook-web | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-
by-default.sh
guestbook-web | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/ngin
x/conf.d/default.conf
guestbook-web | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/ng
inx/conf.d/default.conf
guestbook-web | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.
envsh
guestbook-web | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-tem
plates.sh
guestbook-web | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-pro
cesses.sh
guestbook-web | /docker-entrypoint.sh: Configuration complete; ready for start up
guestbook-web | 2025/12/26 14:05:04 [notice] 1#1: using the "epoll" event method
guestbook-web | 2025/12/26 14:05:04 [notice] 1#1: nginx/1.29.4
guestbook-web | 2025/12/26 14:05:04 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
```



Code :

Docker -Compse-yml:



```
1  services:
2      # MongoDB Database
3      mongodb:
4          image: mongo:7-jammy
5          container_name: guestbook-db
6          volumes:
7              - mongo_data:/data/db
8          networks:
9              - api_db_net
10
11     # Backend API
12     api:
13         build: ./backend
14         image: backend
15         container_name: guestbook-api
16         environment:
17             - MONGO_URL=mongodb://mongodb:27017
18         ports:
19             - "3000:3000"
20         depends_on:
21             - mongodb
22         networks:
23             - frontend_api_net
24             - api_db_net
25
26     # Frontend
27     web:
28         build: ./frontend
29         image: frontend
30         container_name: guestbook-web
31         ports:
32             - "8080:80"
33         depends_on:
34             - api
35         networks:
36             - frontend_api_net
37
38     networks:
39         frontend_api_net:
40         api_db_net:
41
42     volumes:
43         mongo_data:
44
```

Backend Docker File :



```
1 FROM node:lts-alpine3.23
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm install
5 COPY server.js .
6 EXPOSE 3000
7 CMD ["npm", "start"]
```

Frontend Docker File :



```
1 FROM nginx:trixie
2 COPY index.html /usr/share/nginx/html/
3 EXPOSE 80
```